

GETTING STARTED WITH LOCAL LLMS

O'Reilly Live Training — Quick Reference Cheatsheet

Gemma 4 · Ollama · RAG · Tool Calling · Agents · Fine-Tuning · 2026 Edition

SETUP & ENVIRONMENT

Environment (conda)

```
conda create -n oreilly-llama3
python=3.10
conda activate oreilly-llama3
pip install -r requirements/
requirements.txt
cp .env.example .env      # add OpenAI
key

# Jupyter
pip install jupyter
python3 -m ipykernel install \
  --user --name=oreilly-llama3
jupyter lab
```

Install Ollama & Pull Models

```
# Install: https://ollama.com/download
ollama pull gemma4:26b      # 8GB VRAM ★
ollama pull gemma4:e4b     # CPU-only
ollama pull gemma4:31b     # 20GB VRAM
ollama serve                #
localhost:11434
ollama list                 # show
installed
ollama run gemma4           #
interactive chat
ollama rm modelname        # remove
model
```

MODULE 1 — OLLAMA QUICKSTART

Python API

```
import ollama

# Simple chat
response = ollama.chat(
    model="gemma4",
    messages=[{
        "role": "user",
        "content": "Explain RAG in 2
sentences."
    }]
)
print(response['message']['content'])

# Streaming
stream = ollama.chat(
    model="gemma4",
    messages=[{"role": "user",
"content": "Hi"}],
    stream=True
)
for chunk in stream:
    print(chunk['message']['content'],
end='')
```

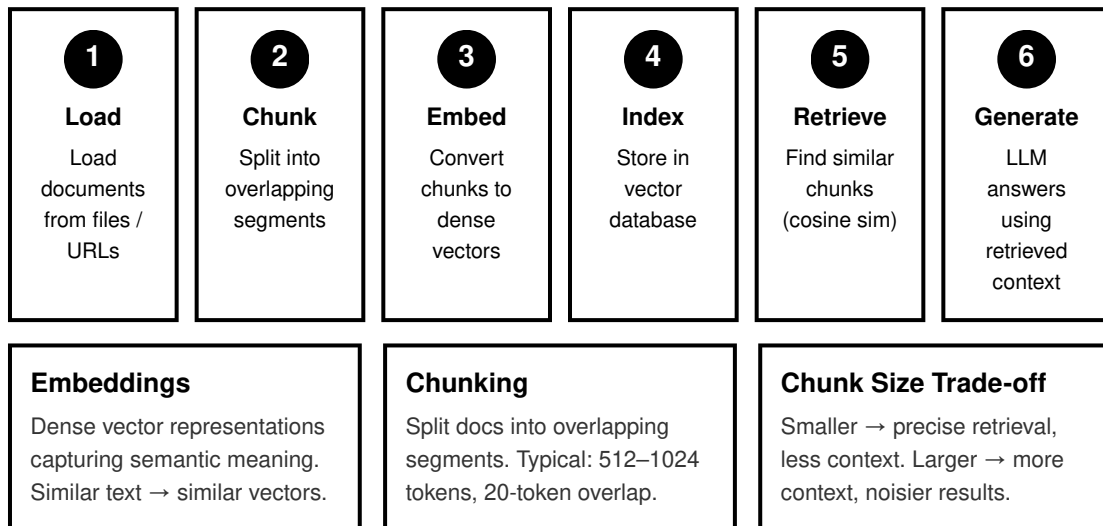
Reusable Helper

```
import ollama

def chat(prompt, model="gemma4",
        system=None):
    messages = []
    if system:
        messages.append({
            "role": "system",
            "content": system
        })
    messages.append({
        "role": "user",
        "content": prompt
    })
    resp = ollama.chat(
        model=model,
        messages=messages
    )
    return resp['message']['content']

# Usage
answer = chat("What is LoRA?")
print(answer)
```

MODULE 2 — INTRODUCTION TO RAG



MODULE 3 — LOCAL RAG WITH GEMMA 4 + LLAMA INDEX

Build Index & Query

```
from llama_index.llms.ollama import Ollama
from llama_index.embeddings.huggingface \
    import HuggingFaceEmbedding
from llama_index.core import (
    VectorStoreIndex,
    SimpleDirectoryReader, Settings
)

Settings.llm = Ollama(
    model="gemma4",
    request_timeout=120.0
)
Settings.embed_model = HuggingFaceEmbedding(
    model_name="BAAI/bge-small-en-v1.5"
)
docs = SimpleDirectoryReader("data/").load_data()
index = VectorStoreIndex.from_documents(docs)
engine = index.as_query_engine(
    similarity_top_k=3
)
response = engine.query("What is LoRA?")
```

Inspect Sources & Persist

```
# View retrieved source chunks
for node in response.source_nodes:
    print(node.text[:200])
    print(node.metadata)

# Persist index to disk
index.storage_context.persist(
    persist_dir="./storage"
)

# Reload from disk
from llama_index.core import (
    StorageContext,
    load_index_from_storage
)
ctx = StorageContext.from_defaults(
    persist_dir="./storage"
)
index = load_index_from_storage(ctx)
```

RAG Stack

Component	Tool	Why
LLM	Ollama (gemma4)	Local, no API key, easy model management
Embeddings	BAAI/bge-small-en-v1.5	384-dim, fast, runs locally (HuggingFace)
Orchestration	LlamaIndex	Handles chunking, indexing, retrieval
Vector Store	In-memory / ChromaDB	In-memory for dev, ChromaDB for persistence

MODULE 4 — TOOL CALLING WITH OLLAMA

Define & Register Tools

```
import ollama

def get_weather(city: str) -> str:
    return f"Weather in {city}: 22°C sunny"

tools = [{
    "type": "function",
    "function": {
        "name": "get_weather",
        "description": "Get weather for a city",
        "parameters": {
            "type": "object",
            "properties": {
                "city": {"type": "string"}
            },
            "required": ["city"]
        }
    }
}]

response = ollama.chat(
    model="gemma4",
    messages=[{
        "role": "user",
        "content": "Weather in Paris?"
    }],
    tools=tools
)
```

Execute Tool Calls

```
tools_map = {"get_weather":
get_weather}

if response.message.tool_calls:
    for tc in
response.message.tool_calls:
        fn =
tools_map[tc.function.name]
        result =
fn(**tc.function.arguments)

        # Feed result back to model
        messages = [
            response.message,
            {
                "role": "tool",
                "content": str(result),
                "name":
tc.function.name
            }
        ]
        final = ollama.chat(
            model="gemma4",
            messages=messages
        )
        print(final.message.content)
```

MODULE 5 — LOCAL AGENTS: STRUCTURED OUTPUTS → AGENT LOOP

Structured Outputs (Pydantic)

```
from pydantic import BaseModel
import ollama

class Task(BaseModel):
    title: str
    priority: str # high/medium/low
    estimated_hours: int

response = ollama.chat(
    model="gemma4",
    messages=[{
        "role": "user",
        "content": "Create a task to
build login page"
    }],
    format=Task.model_json_schema()
)
task = Task.model_validate_json(
    response.message.content
)
print(task.title, task.priority)
```

Agent Concepts Summary

```
# Agent Loop pattern:
# User → LLM decides tool → Execute
tool
# → Feed result back → LLM final
response

# Concept          Ollama Feature
# -----
# -----
# Structured
format=Schema.model_json_schema()
# Outputs
#
# Tool Calling  tools=[{type,
function...}]
#
#               (LLM decides which to
call)
#
# Agent Loop    Combine both:
#               1. Send messages +
tools
#               2. Execute tool_calls
#               3. Append tool results
#               4. Re-call chat() until
#               no tool_calls remain
```

MODULE 6 — AGENTIC RAG: REACT PATTERN

Simple ReAct Agent (Reason → Act → Observe loop)

```
import ollama

class SimpleReActAgent:
    def __init__(self, model="gemma4", tools=None, tools_map=None, max_turns=10):
        self.model, self.tools = model, tools or []
        self.tools_map, self.max_turns = tools_map or {}, max_turns
        self.messages = []

    def run(self, user_input: str) -> str:
        self.messages.append({"role": "user", "content": user_input})
        for turn in range(self.max_turns):
            response = ollama.chat(model=self.model, messages=self.messages,
tools=self.tools)
            if not response.message.tool_calls:
                return response.message.content # No tool call = final
answer
            self.messages.append(response.message)
            for tc in response.message.tool_calls: # Execute each tool
                result = self.tools_map[tc.function.name](**tc.function.arguments)
                self.messages.append({"role": "tool", "content": str(result),
"name": tc.function.name})
            return "Max turns reached."

# Extend by: adding tools, adjusting max_turns, custom system prompt, adding
memory
```

ReAct Loop: Reason → Act → Observe (repeat until done)

- **Turn 1:** Agent receives query, calls tool (e.g. `find_pdfs`) to discover resources
- **Turn N:** Agent receives tool results, may call more tools or synthesise final answer
- **Done:** When LLM returns a response with no `tool_calls`

MODULE 7 — USEFUL LOCAL AGENTS: WHEN LOCAL VS CLOUD

Local Agent ✓ Use When...

- Privacy matters — data must stay on device
- Cost control — no per-token billing
- Single-tool chains with deterministic output
- Private document analysis
- Offline / air-gapped environments
- Running Hermes Agent (local TUI + memory)

Cloud Agent ✓ Use When...

- Long-context synthesis (>64K tokens)
- Multi-hop reasoning chains (complex logic)
- Code edits beyond ~200 LOC
- Ambiguous planning needing world knowledge
- Tasks needing latest internet data
- High-quality generation is critical

MODULE 8 — FINE-TUNING (LORA, QLoRA, PEFT)

Methods

Method	Description
SFT	Supervised Fine-Tuning: instruction→response pairs
RLHF	Reinforcement Learning from Human Feedback
PEFT	Update only a small subset of parameters
LoRA	Train small adapter matrices A,B (10,000× fewer params)
QLoRA	4-bit quantized base + LoRA adapters in higher precision

When to Fine-Tune

- **Cost:** Replace expensive GPT-4 with cheaper fine-tuned model
- **Latency:** Smaller model = faster responses
- **Consistency:** Model reliably follows specific format/style

Start with prompting. Fine-tune only when product shape is finalized and use case is clear.

Hardware (QLoRA min.):

- GPU: 24GB VRAM (RTX 4090)
- RAM: 32GB+ system memory
- Storage: 100GB+ SSD free

Tools:

AutoTrain · Unsloth · Together AI · Axolotl · LLaMA-Factory

MODULE 9 — QUANTIZATION

Precision Levels

Precision	Relative Size	Primary Use	Notes
FP32	Baseline (1×)	Training	Full precision, maximum memory
FP16 / BF16	2× smaller	Fine-tuning	Default for most training pipelines
INT8	4× smaller	Inference	Minimal quality loss
INT4 (GGUF/ AWQ)	8× smaller	Consumer GPU / CPU	Fits 2× larger models on same VRAM

Q4 Key Insight

INT4 (Q4) quantization lets you fit models **2× larger than your VRAM** on the same hardware. Example: Gemma4 26B (MoE, Q4) runs on **8GB VRAM** — it activates only 3.8B params per token.

MODULE 10 — BEST LOCAL MODELS 2026

Hardware Tiers — Recommended Models

Tier	Hardware	Top Pick	Ollama Tag	Strength
1	CPU, 8–16GB RAM	Qwen3 4B ★	<code>qwen3:4b</code>	General, multilingual, 2.5GB disk
1	CPU, multimodal	Gemma4 E4B	<code>gemma4:e4b</code>	Image + audio on CPU, Apache 2.0
1	CPU, reasoning	Phi-4 Mini	<code>phi4-mini</code>	Best math/reasoning at 3.8B (MIT)
2	8GB VRAM	Gemma4 26B ★ (course default)	<code>gemma4:26b</code>	Multimodal, #3 Chatbot Arena, MoE (3.8B active)
2	10–16GB VRAM	Phi-4 14B ★	<code>phi4</code>	Matches Llama 3.3 70B reasoning at 5× smaller (MIT)
2	8GB VRAM	Qwen3 8B	<code>qwen3:8b</code>	Coding, multilingual, hybrid thinking mode
3	20–24GB VRAM	Qwen3.6 27B ★	<code>qwen3.6:27b</code>	Best coding+agents, SWE-bench 77.2%, 256K ctx
3	20GB VRAM	DeepSeek-R1 32B	<code>deepseek-r1:32b</code>	Best chain-of-thought reasoning (MIT)
3	20GB VRAM	Gemma4 31B	<code>gemma4:31b</code>	Best multimodal quality for 24GB VRAM
4	48GB+ VRAM	DeepSeek-R1 70B	<code>deepseek-r1:70b</code>	Frontier reasoning, MIT, single A100 80GB

Tier 1 — Quick Start Commands (CPU)

```
ollama pull qwen3:4b          # general (2.5GB)
ollama pull phi4-mini         # reasoning/math
ollama pull gemma4:e4b        # multimodal CPU
ollama pull gemma4:e2b        # ultra-light (≤8GB)
```

Tier 2–3 — GPU Commands

```
ollama pull gemma4:26b        # 8GB VRAM ★
ollama pull phi4              # 10-16GB VRAM
ollama pull qwen3:8b          # 8GB VRAM coding
ollama pull qwen3.6:27b       # 24GB agents
ollama pull deepseek-r1:32b   # 24GB reasoning
```

COURSE KEY TAKEAWAYS

- **Ollama** — One command to run any open-weight model locally (`ollama pull + ollama.chat()`)
- **RAG** — Load → Chunk → Embed → Index → Retrieve → Generate: give LLMs access to your own documents
- **Tool Calling** — Pass `tools=[...]` to Ollama; LLM decides which function to call and with what args
- **Agents** — Structured outputs (Pydantic) + tool calling + a loop = a capable local agent
- **ReAct** — Reason → Act → Observe loop; keep looping until no `tool_calls` in response
- **Local vs Cloud** — Local wins on privacy and cost; cloud wins on long context, complex reasoning, and latest data
- **Fine-Tuning** — Use QLoRA (4-bit base + LoRA adapters) for consumer hardware; only fine-tune when task is well-defined
- **Quantization** — INT4 (Q4) fits 8× more model into same VRAM at minimal quality cost
- **Gemma 4** — Course default: multimodal, Apache 2.0, #3 Chatbot Arena, all hardware tiers covered